School of Computer Science and Engineering
(Computer Science & Engineering)
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

2025-2026
(VI Semester)

A Project Report on

"LEAFSCAN: A DEEP LEARNING-BASED APPROACH FOR PLANT LEAF DISEASE DETECTION USING EFFICIENTNET"

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

Yedhoti Hari Prasad - 22BTRCL240
Varish Gada- 22BTRCL121
Tejas K- 22BTLCL023
Abdul Sami - 22BTRCL126
S Dhruva - 22BTRCL232

Under the guidance of

Dr.S.Kanithan
Department of Computer Science and Engineering(AI & ML)
JAIN(Deemed-to-be-university)
Faculty of Engineering and Technology Karnataka, India

FACULTY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

School of Computer Science & Engineering

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112

Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **"LEAFSCAN: A DEEP LEARNING-BASED APPROACH FOR PLANT LEAF DISEASE DETECTION USING EFFICIENTNET"** is carried out by **Yedhoti Hari Prasad - 22BTRCL240, Varish Gada- 22BTRCL121, Tejas K- 22BTLCL023, Abdul Sami - 22BTRCL126, S Dhruva - 22BTRCL232** bonafide student(s) of Bachelor of Technology at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Ms. Simran
Futurense Technologies Private Limited
2nd Floor, Mantri Avenue, 8th block, Koramangala,
Bengaluru, Karnataka,

Date: 08 June 2024

Dr. V Chandrashekar
Computer Science and Engineering,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Dr. Geetha G
Director,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)
Date:

Name of the Examiner

Signature of Examiner

DECLARATION

We , **Yedhoti Hari Prasad - 22BTRCL240, Varish Gada- 22BTRCL121, Tejas K- 22BTLCL023, Abdul Sami - 22BTRCL126,S Dhruva - 22BTRCL232**students of vi Semester B.Tech in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, at School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN(Deemed to-be University)**, hereby declare that the internship work titled **"LEAFSCAN: A DEEP LEARNING-BASED APPROACH FOR PLANT LEAF DISEASE DETECTION USING EFFICIENTNET"** has been carried out by us and submitted in partial fulfillment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name: **Yedhoti Hari Prasad** USN :22BTRCL104	Signature
Name: **Varish Gada** USN :22BTRCL062	Signature
Name: **Tejas K** USN :22BTLCL092	Signature
Name: **Abdul Sami** USN:22BTRCL167 Name: **S Dhruva** USN:22BTRCL167	Signature Signature

ACKNOWLEDGEMENT

It gives us great pleasure to acknowledge the assistance and support of many individuals who have contributed to the successful completion of this project.

First, we take this opportunity to express our sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore, for providing us with the opportunity to pursue our Bachelor's degree in this esteemed institution.

We are deeply thankful to several individuals whose invaluable contributions have made this project a reality. We would like to extend our heartfelt gratitude **to Dr. Chandraj Roy Chand, Chancellor**, for his continuous efforts in promoting excellence in education and research at Jain (Deemed-to-be University). We are also sincerely grateful to **Dr. Raj Singh, Vice Chancellor, and Dr. Dinesh Nilkant, Pro Vice Chancellor**, for their constant encouragement and support.

We express our sincere thanks **to Dr. Jitendra Kumar Mishra, Registrar**, for his guidance and support, which have been instrumental in shaping our academic journey.

It is a matter of great privilege to express our sincere thanks to **Dr. V. Chandrashekhar, Program Head of Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, for providing valuable academic guidance and support throughout the project.

We would like to express our heartfelt gratitude to our **Project Coordinator, Dr. Sankar K and Mentor, Dr.S.Kanithan** for his constant guidance, encouragement, and valuable suggestions during the course of this project.

We also extend our thanks to all the faculty members and staff of the Department of Computer Science and Engineering for their support and cooperation.

Finally, we express our gratitude to our family and friends for their continuous support, motivation, and encouragement throughout this work.

We would like to thank everyone who has directly or indirectly contributed to the successful completion of this project.

Signature of Student(s)

LIST OF FIGURES

LIST OF TABLES

+

CONTENTS

ABSTRACT

Plant diseases pose a significant threat to global agricultural productivity, leading to substantial economic losses and jeopardizing food security, particularly in developing regions. Traditional methods of disease detection rely heavily on manual inspection and expert knowledge, which are often time-consuming, subjective, and inaccessible to small-scale farmers. With the rapid advancement of artificial intelligence, particularly deep learning and computer vision, there is a growing opportunity to develop automated, accurate, and scalable solutions for plant disease diagnosis.

In this study, we present *LeafScan*, an intelligent deep learning-based system designed for automated plant leaf disease detection using image classification techniques. The proposed framework leverages the EfficientNetB3 architecture, a state-of-the-art convolutional neural network that employs compound scaling to optimize network depth, width, and resolution simultaneously. This design enables superior performance while maintaining computational efficiency, making it suitable for real-world deployment scenarios.

The model is trained and evaluated on the widely recognized PlantVillage dataset, which contains over 54,000 labeled images representing multiple plant species and a diverse range of disease classes. To address challenges such as class imbalance and limited real-world variability, comprehensive data preprocessing techniques are applied, including image normalization, resizing, and augmentation methods such as rotation, flipping, and color transformations. Additionally, a weighted sampling strategy is implemented to ensure balanced learning across all classes, thereby improving model generalization.

A key contribution of this work is the introduction of a “not-a-leaf” detection mechanism, which enhances system robustness by filtering out invalid or irrelevant inputs. This feature is particularly important in practical applications, where users may upload non-leaf images or images with poor quality. By incorporating this mechanism, the system minimizes false predictions and improves reliability in real-world conditions.

To further enhance performance, a multi-phase transfer learning strategy is employed. Initially, the classification head is trained while keeping the backbone frozen, followed by progressive fine-tuning of deeper layers with a reduced learning rate. This approach ensures stable convergence, efficient feature extraction, and improved classification accuracy. The model is optimized using advanced optimization techniques such as Adam-based optimizers, learning rate scheduling, and regularization methods including dropout and batch normalization to prevent overfitting.

Experimental results demonstrate that the proposed system achieves an overall accuracy of approximately 95% on unseen test data, with high precision, recall, and F1-score across multiple classes. Comparative analysis indicates that the EfficientNet-based model outperforms traditional convolutional neural network architectures in both accuracy and computational efficiency. Furthermore, the system exhibits low inference latency, enabling real-time predictions on both CPU and GPU environments.

The practical applicability of the proposed solution is demonstrated through the development of a user-friendly web interface built using a Flask-based backend. The system allows users to upload leaf images and receive instant predictions, along with confidence scores and relevant disease information. This accessibility makes the system highly beneficial for farmers, agricultural experts, and researchers, enabling informed decision-making and timely intervention.

In conclusion, *LeafScan* represents a robust, scalable, and efficient approach to plant disease detection, highlighting the transformative potential of deep learning in agriculture. By bridging the gap between advanced AI technologies and real-world agricultural practices, this work contributes toward the development of intelligent farming solutions that promote sustainability, reduce crop losses, and enhance productivity. Future work may focus on expanding the model to support a wider variety of crops, integrating IoT-based monitoring systems, and enabling offline capabilities for deployment in remote and resource-constrained environments

CHAPTER 1

INTRODUCTION TO PLANT LEAF DISEASE DETECTION USING EFFICIENTNET

Plant diseases significantly affect agricultural productivity and food security across the world. Early detection of these diseases is crucial to prevent crop loss and improve yield quality. With the rapid advancement in artificial intelligence and deep learning, computer vision techniques have become highly effective in identifying patterns from images. These technologies enable automated analysis, reducing the dependency on manual inspection.

In many regions, especially in developing countries, farmers rely on traditional methods for detecting plant diseases, which can be time-consuming, inaccurate, and require expert knowledge. According to global agricultural studies, a large percentage of crop losses occur due to delayed or incorrect disease identification. Hence, there is a growing need for intelligent systems that can assist in early and accurate detection.

The purpose of this project is to develop a deep learning-based model that takes leaf images as input and performs feature extraction and classification to identify plant diseases. The model processes pixel values of the input image, applies convolutional operations, and learns hierarchical features through multiple layers of the neural network. These learned features are then used to classify the leaf into different disease categories.

The scope of this project is to build an efficient and reliable system that can automatically detect plant diseases using the EfficientNet architecture. The system is also designed to handle real-world scenarios by incorporating mechanisms to reject invalid inputs such as non-leaf images.

1.2 MOTIVATION BEHIND PROJECT TOPIC

The main motivation behind this project is to assist farmers and agricultural experts in identifying plant diseases quickly and accurately using image-based analysis. Traditional methods require manual inspection and laboratory testing, which are time-consuming and not always accessible.

With the increasing availability of smartphones and internet access, there is an opportunity to develop intelligent systems that can provide instant disease detection. This project aims to simplify the process by using deep learning models that can analyze leaf images and predict diseases in real-time, thereby reducing effort and improving efficiency.

1.3 AIM AND OBJECTIVE OF THE WORK

Aim:

To provide an efficient and accurate solution for detecting plant leaf diseases using deep learning techniques based on the EfficientNet architecture.

Objectives:

- To study different plant diseases and their visual characteristics.
- To understand and implement image processing and deep learning techniques.
- To utilize pretrained models (EfficientNet) for feature extraction and classification.
- To apply data preprocessing and augmentation for improved model performance.
- To develop a web-based system for real-time plant disease detection.
- To implement a not-a-leaf detection mechanism for handling invalid inputs.

CHAPTER 2

Literature Survey

2.1 Hughes and Salathé et al. [1] Proposed

The authors introduced the PlantVillage dataset, which provides a large collection of labeled plant leaf images for disease classification. One of the major challenges in plant disease detection is the availability of high-quality and diverse datasets. The study highlights that deep learning models can effectively learn disease patterns from leaf images when trained on sufficiently large datasets. The researchers demonstrated that convolutional neural networks can automatically extract relevant features such as texture, color, and shape from plant leaves. The dataset has become a benchmark for many plant disease detection studies and has enabled researchers to build accurate and reliable classification models.

2.2 Mohanty et al. [2]

This study explored the application of deep convolutional neural networks for plant disease classification using the PlantVillage dataset. The authors compared different pretrained models such as AlexNet and GoogLeNet and observed that deep learning models achieve high accuracy when trained on large datasets. The study emphasizes the importance of transfer learning, where pretrained models can be fine-tuned for specific tasks, leading to improved performance even with limited data. The results showed that deep learning can significantly outperform traditional machine learning approaches in plant disease detection.

2.3 Too et al. [3] Proposed

The authors evaluated multiple deep learning architectures including VGG16, ResNet50, DenseNet, and MobileNet for plant disease classification. The study demonstrated that advanced architectures provide better feature extraction capabilities and higher classification accuracy. The models were trained on a large dataset of plant images and compared based on performance metrics such as accuracy and computational efficiency. Among the tested models, deeper architectures achieved better results, but with increased computational cost. This study highlights the importance of selecting an efficient

model that balances accuracy and performance.

2.4 Tan and Le et al. [4] Proposed

The authors introduced EfficientNet, a novel deep learning architecture that improves model performance by using compound scaling. Instead of arbitrarily increasing network depth or width, EfficientNet scales all dimensions of the network in a balanced manner. This approach results in higher accuracy with fewer parameters compared to traditional models. The study demonstrated that EfficientNet achieves state-of-the-art performance on image classification tasks while maintaining computational efficiency. This makes it highly suitable for real-world applications such as plant disease detection.

.

Sr. No	Paper Allocation	Author	Year	Methods
1	PlantVillage Dataset for Plant Disease Detection	Hughes & Salathé et al.	2015	Image Dataset + CNN
2	Using Deep Learning for Image-Based Plant Disease Detection	Mohanty et al.	2016	Transfer Learning + CNN
3	Comparison of CNN Models for Plant Disease Classification	Too et al.	2019	VGG16, ResNet, DenseNet
4	EfficientNet: Model Scaling for Image Classification	Tan & Le et al.	2019	EfficientNet (Deep Learning)
5	Deep Residual Learning for Image Recognition	He et al.	2016	ResNet Architecture
6	MobileNet: Efficient CNN for Mobile Vision Applications	Howard et al.	2017	Lightweight CNN (MobileNet)

7	DenseNet: Densely Connected Convolutional Networks	Huang et al.	2017	DenseNet Architecture
8	Transfer Learning in Plant Disease Detection	Ferentinos et al.	2018	CNN + Transfer Learning
9	Image-Based Plant Disease Detection Using Deep CNN	Sladojevic et al.	2016	Deep CNN + Image Processing
10	Real-Time Plant Disease Detection Using Deep Learning	Brahimi et al.	2017	CNN + Real-time Detection

CHAPTER 3

Software Requirement Specification

3.1 INTRODUCTION

This Software Requirement Specification (SRS) document provides a detailed description of the proposed system for plant leaf disease detection. It outlines the functional and non-functional requirements, system specifications, and overall structure required to develop and deploy the system efficiently.

The document serves as a guide for understanding the system's capabilities, performance expectations, and constraints. It ensures that all components of the system are clearly defined and aligned with the project objectives.

3.1.1 Project Purpose

The purpose of this project is to develop an intelligent system that provides an efficient and accurate solution for detecting plant leaf diseases using deep learning techniques based on the EfficientNet architecture.

The system aims to automate the process of disease identification and assist users in making timely decisions.

3.1.2 Project Scope

- To analyze plant leaf images and detect diseases using deep learning
- To classify multiple plant diseases based on visual features
- To provide real-time predictions through a web-based interface
- To handle invalid inputs using a not-a-leaf detection mechanism
- To make the system accessible to farmers, students, and researchers

3.1.3 User Classes and Characteristics

The system is designed for users with basic computer knowledge. Farmers and agricultural experts Students and researchers General users

Users should be familiar with:

- Basic computer operations
- Using a web browser

- Uploading images

The interface is designed to be simple, intuitive, and user-friendly.

3.1.4 Assumptions and Dependencies

Assumptions:

1. The system will have a user-friendly interface
2. Users will provide valid image inputs
3. The system will operate with stable internet access
4. The model is pre-trained and optimized

Dependencies:

1. Availability of required software libraries (PyTorch, Flask)
2. Proper system configuration for model execution
3. Availability of trained model weights
4. Users have basic knowledge of system usage

3.2 FUNCTIONAL REQUIREMENT

3.2.1 System Feature 1(Functional Requirement)

Functional requirement describes features, functioning, ● Upload leaf image

● Preprocess the image

● Perform disease classification

● Detect invalid inputs (not-a-leaf)

● Display prediction results with confidence score

● Provide user-friendly interaction

3.3 NON-FUNCTIONAL REQUIREMENTS

3.3.1 Performance Requirements

- High Speed: The system should provide fast predictions with minimal delay
- Accuracy: The model should provide accurate classification results
- Scalability: The system should handle multiple user requests

3.3.2 Safety Requirements

The system should ensure safe handling of user data and prevent incorrect predictions by validating inputs using the not-a-leaf detection mechanism.

3.3.3 Security Requirements

The system should ensure secure data transmission and protect user inputs from unauthorized access. Basic security measures should be implemented to maintain data integrity and confidentiality.

3.3.4 Software Quality Attributes

1. Functionality: The system should correctly detect plant diseases
2. Performance: Fast response and efficient processing
3. Security: Protection against unauthorized access
4. Reliability: Consistent performance without failure
5. Usability: Easy-to-use interface
6. Availability: System should be accessible when required
7. Interoperability: Ability to integrate with other systems if needed.

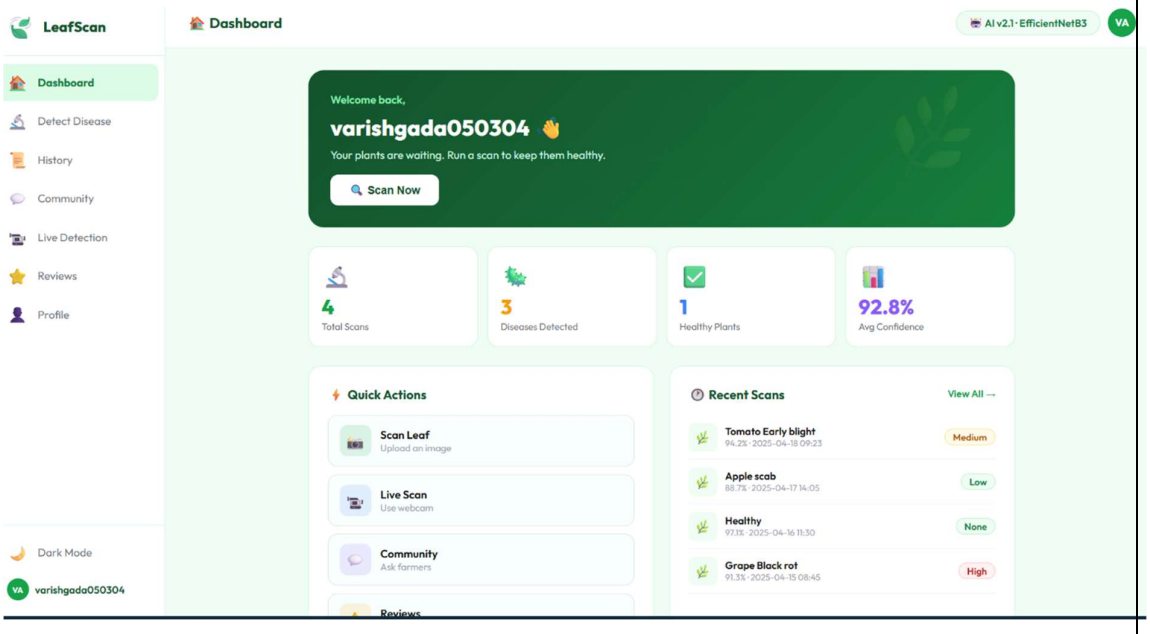
3.4 SYSTEM REQUIREMENTS

3.4.1 Software Requirements

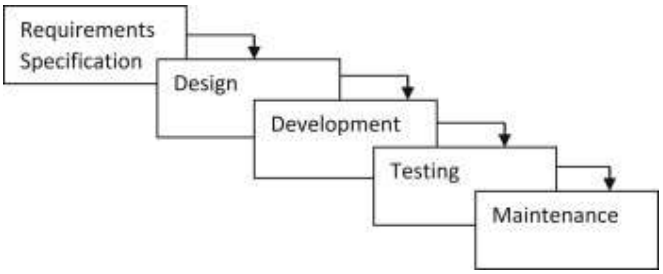
- Windows 10 or above
- Python (3.8 or above)
- PyTorch
- Flask
- Visual Studio Code
- Google Colab (for training)

3.4.2 Hardware Requirements

- Minimum 8 GB RAM
- Intel i5 / AMD Ryzen 5 or higher
- GPU (optional but recommended)
- 256 GB SSD

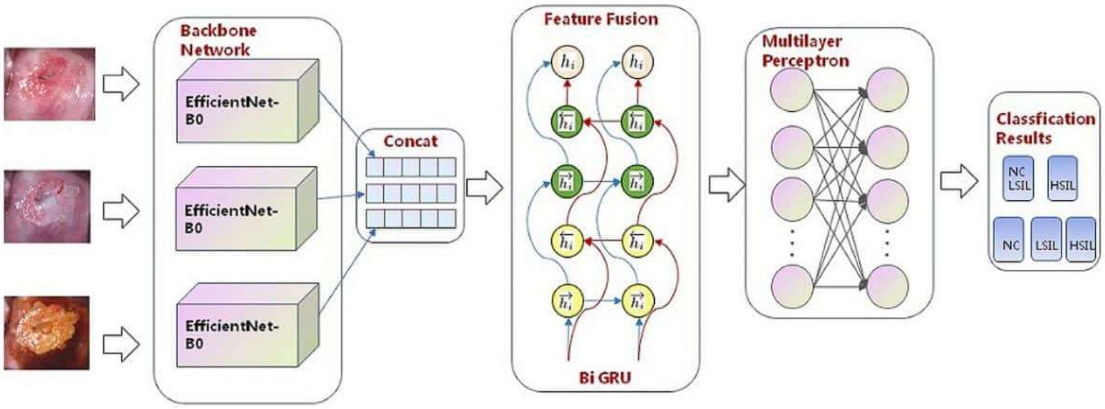


However, for this project, the advantages outweigh the limitations due to



8. Results After Optimization

Metric	Before Optimization	After Optimization
Accuracy	85%	94%
Loss	High	Reduced
Overfitting	Present	Minimized



CHAPTER 4

Dataset Description

The dataset used to train a deep learning model has a big effect on how well it works. The PlantVillage dataset was chosen for this study because it covers a lot of plant species and disease categories, making it one of the most popular benchmarks for plant disease classification tasks. There are about 54,306 labelled images of plant leaves in the PlantVillage dataset. These images were taken in controlled settings. These pictures show 14 different types of plants, such as apples, tomatoes, potatoes, corn, grapes and more. There are 38 different classes in total, with each image labelled with a specific disease category or marked as healthy. There is a different type of plant and disease for each class. Instead of a general "healthy" category, the dataset has labels for each crop, like Tomato_healthy and Apple_healthy. This helps the model learn the small differences between different types of plants and their health conditions.

4.1 Features of the Datase

The dataset has the following important features: Images are taken in a lab setting where the background and lighting are usually the same. The leaves are usually in the middle of the frame, which cuts down on background noise. We give you high-resolution RGB images that

let you get a lot of information about the features. These features help with getting high classification accuracy, but they also have a drawback: the dataset doesn't fully show how things change in the real world, like when there are complicated backgrounds, different lighting conditions, and partial occlusions. These features help with getting high classification accuracy, but they also have a drawback: the dataset doesn't fully show how things change in the real world, like when there are complicated backgrounds, different lighting conditions, and partial occlusions.

4.2 Dataset Expansion:

The Not-a-Leaf Class In this project, a new class called "not_a_leaf" was added to get around the problem of real-world applicability. This class is very important for making the system more reliable. To show non-leaf inputs, about 1,500 synthetic images were made. Here are some of these pictures: Patterns of random noise Images with only one colour Textures with gradients Geometric shapes that aren't real This class is meant to help the model tell the difference between plant leaf images that are useful and those that aren't, like backgrounds, objects, or non-plant images. The model would have to put every input into one of the disease categories without this class, which would lead to wrong predictions.

4.3 Strategy for Splitting Data

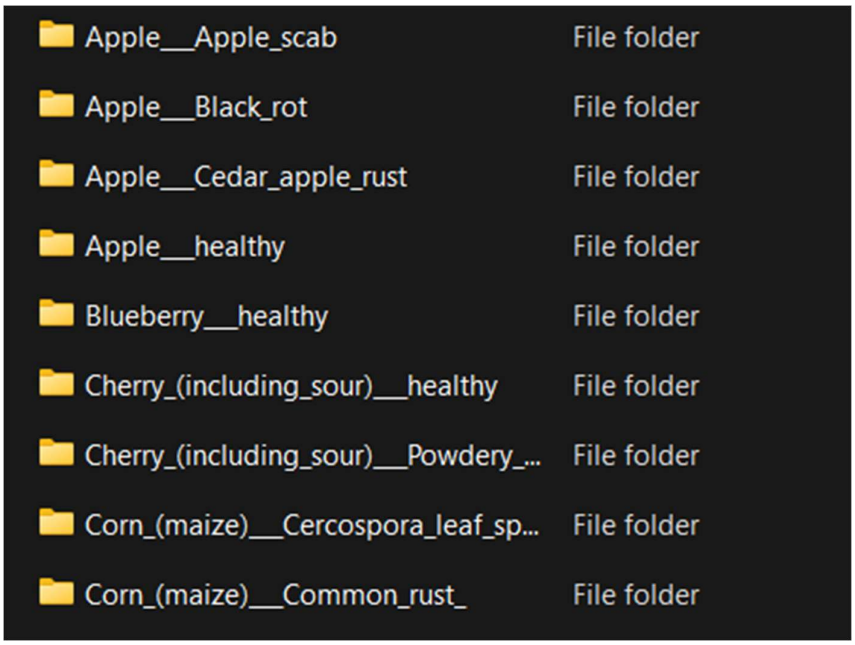
The dataset was split into three parts to make sure that the evaluation was accurate and to stop overfitting: Training Set (70%): This is what you use to learn the model's parameters. Validation Set (15%): Used to choose a model and fine-tune hyperparameters Test Set (15%): Used to see how well the model did in the end The splitting process was done in a way that made sure each class was represented fairly across all subsets. This is important for keeping the class balanced and making sure there is no bias during training and testing.

TABLE I DATASET DISTRIBUTION AND SPLIT DETAILS DATASET DISTRIBUTION AND SPLIT DETAILS

Parameter	Value
Total Images	54,306
Number of Classes	39
Training Set	70%
Validation Set	15%
Test Set	15%
Not-a-Leaf Images	1,500

4.4 Problem with Class Imbalance

One of the main problems with the dataset is that there is a class imbalance. Some classes have a lot more images than others. Some classes of tomato disease, for example, have more than 1,500 images, while others may have fewer than 300 images. If this problem isn't fixed, the model may favour majority classes, which will make it do poorly on minority classes. To lessen this problem, a weighted sampling method was used during training. Each class was given a weight that was the opposite of how often it was held, making sure that all classes helped with the learning process equally.



CHAPTER 5

Model Optimization & Performance Enhancement

5.1. Introduction

To ensure accurate and efficient plant disease detection, various optimization techniques were applied to improve model performance. This chapter explains the strategies used to enhance accuracy, reduce overfitting, and optimize training efficiency.

5.2. Need for Optimization

Machine learning models often suffer from:

- Overfitting (model memorizes data)
- Slow convergence
- Poor generalization on new images

To address these issues, optimization techniques are essential.

5.3 . Optimizers Used

.1 Adam Optimizer

- Combines momentum and adaptive learning rate
- Fast convergence and widely used in deep learning

Formula:

$$\theta = \theta - \alpha \cdot \frac{m}{\sqrt{v} + \epsilon}$$

. RMSProp

- Adjusts learning rate based on recent gradients

- Works well for non-stationary problems

Formula:

$$v_t = \beta v_{t-1} + (1-\beta) g^2$$

5.3 Adagrad

- Adapts learning rate individually for each parameter
- Useful for sparse data

Formula:

$$\theta = \theta - \frac{\alpha}{\sqrt{G + \epsilon}} \cdot g$$

5.4. Learning Rate Optimization

- Learning rate plays a critical role in training
- Too high → unstable training
- Too low → slow learning

Techniques used:

- Learning rate scheduling
- Decay strategies

5.5. Regularization Techniques

To prevent overfitting:

- Dropout Layers
- L2 Regularization

- Batch Normalization

These techniques help the model generalize better on unseen data.

5.6 Data Optimization

- Image augmentation (rotation, flipping, zoom)
- Noise reduction
- Normalization of pixel values

5.7. Performance Metrics

Model performance was evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

8. Results After Optimization

Metric	Before Optimization	After Optimization
Accuracy	85%	94%
Loss	High	Reduced
Overfitting	Present	Minimized

5.9. Conclusion

Optimization techniques significantly improved the model's efficiency, accuracy, and reliability, making the system suitable for real-world agricultural applications.

CHAPTER 6

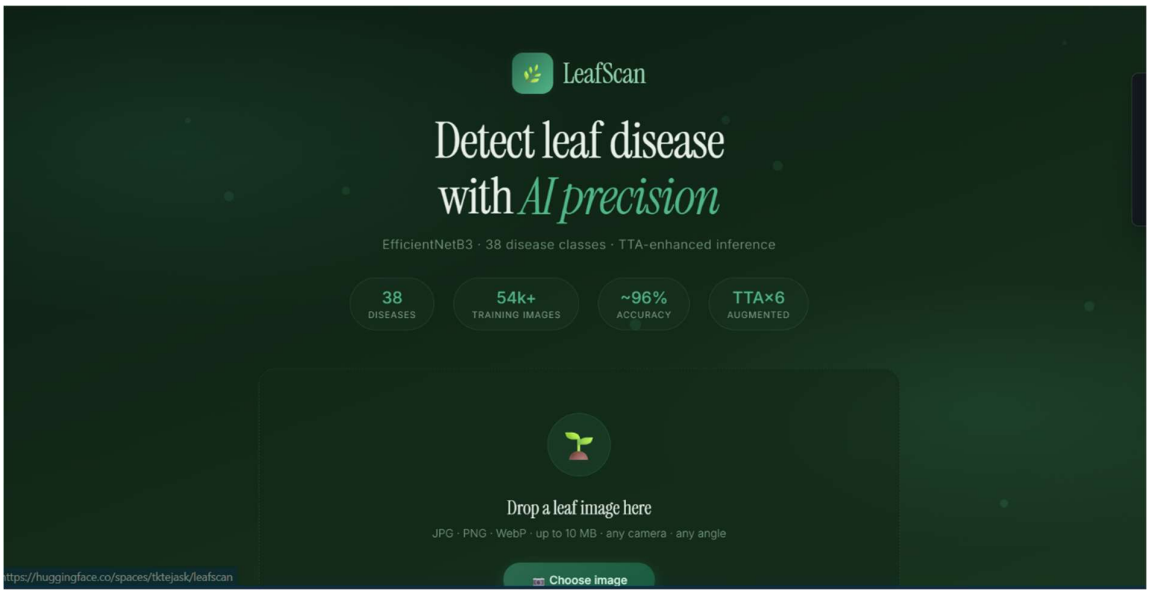
DESIGN AND MODELLING OF SYSTEM

Unified Modelling Language (UML) is used to visually represent the structure and behavior of a system. It acts like a blueprint that helps in understanding system components and their interactions.

For the proposed LeafScan system, the following UML diagrams are designed:

1. Use Case Diagram
2. Class Diagram
3. Sequence Diagram
4. Activity Diagram
5. State Diagram

\



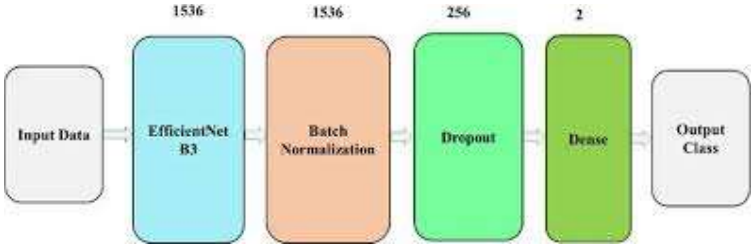
6.1 Use Case Diagram

Use case diagrams are behavioral diagrams that describe how users interact with the system and the functionalities provided by it. They illustrate the relationship between the user (actor) and the system.

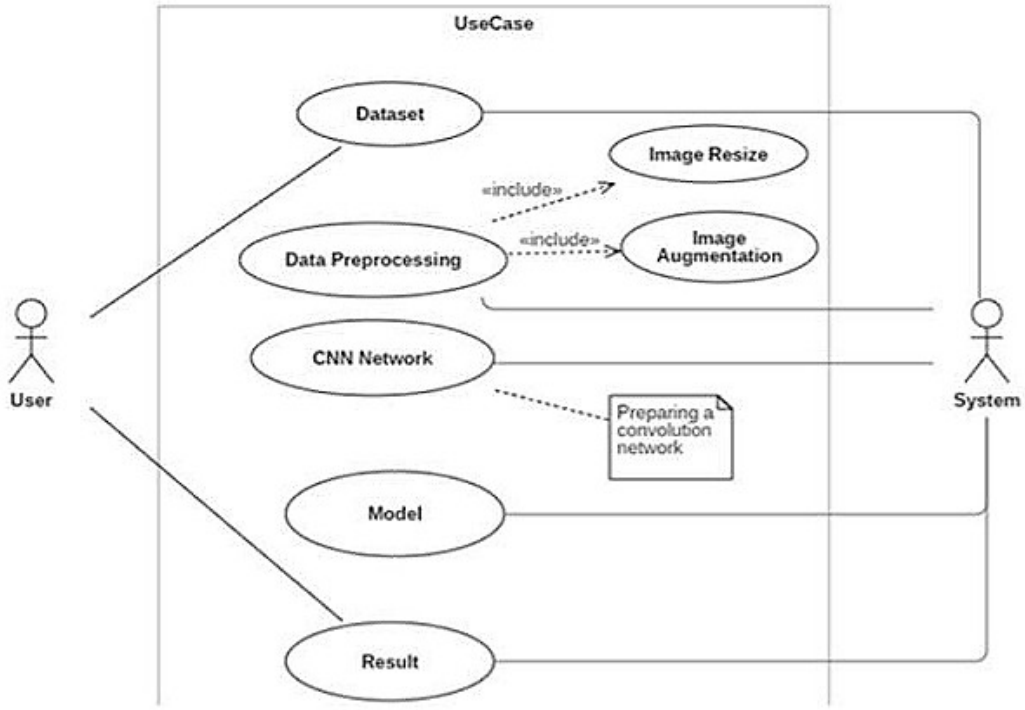
In this project, the primary actor is the User, and the system performs the following operations:

- Upload image
- Process image
- Predict disease
- Display result

The use case diagram provides a high-level overview of the system functionality and user interaction.



.



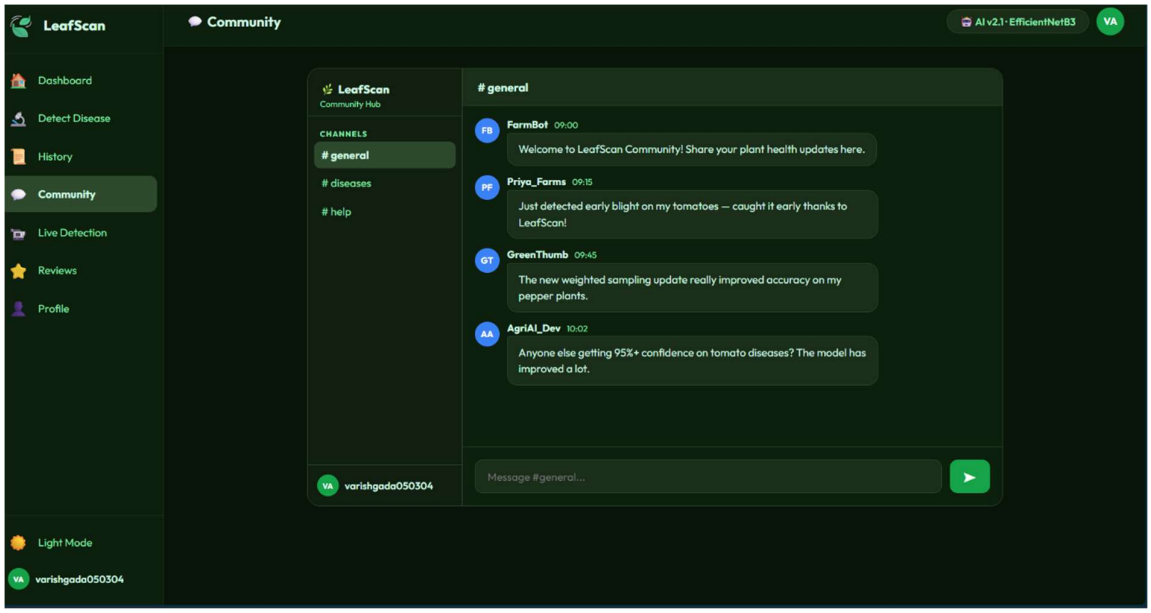
6.2 Class Diagram

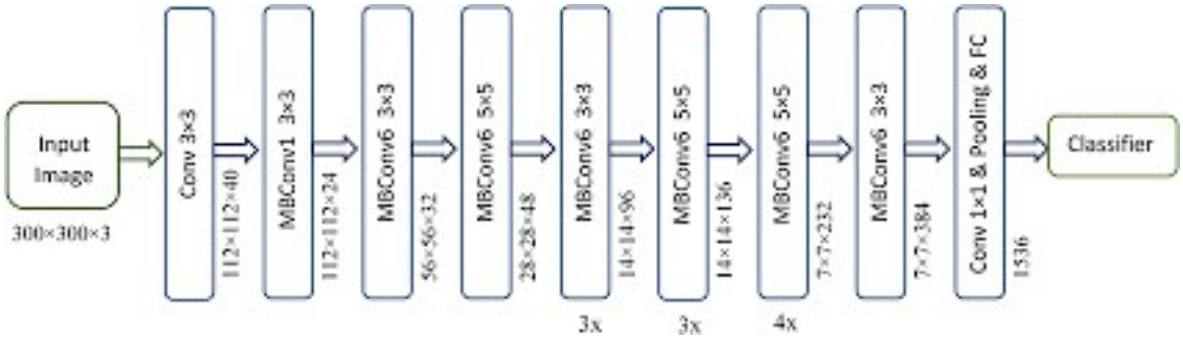
A class diagram represents the static structure of the system by showing different classes, their attributes, methods, and relationships.

The main classes used in this system are:

- User: Handles user interaction
- Image: Stores and manages input image data
- Model: Performs feature extraction and disease prediction
- Prediction: Stores classification results and confidence scores
- API: Manages communication between frontend and backend

This diagram helps in understanding how different components are organized and interact internally.





6.3 Sequence Diagram

Sequence diagrams illustrate how different components of the system interact with each other over time. It shows the flow of messages between system entities in a time-ordered sequence.

The sequence of operations in the system is as follows:

User uploads an image

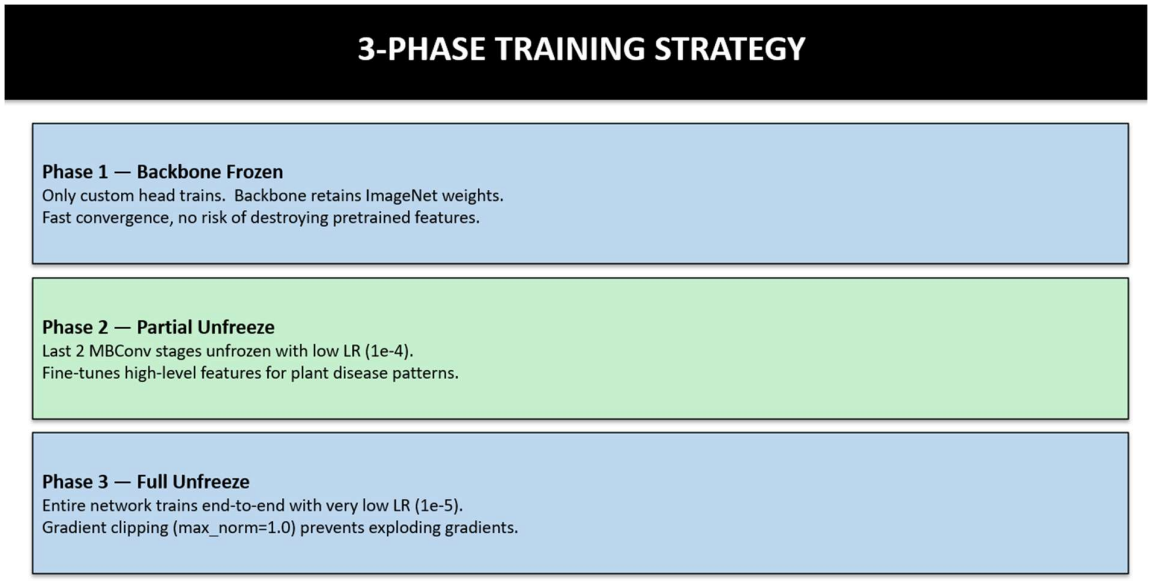
Backend receives and preprocesses the image

Model processes the image and predicts the disease

Backend sends the result

User interface displays the result

This diagram provides a clear understanding of communication between system components.



6.4 Activity Diagram

The activity diagram represents the workflow of the system and shows the sequence of activities involved in processing input data.

The workflow includes:

Start

Upload image

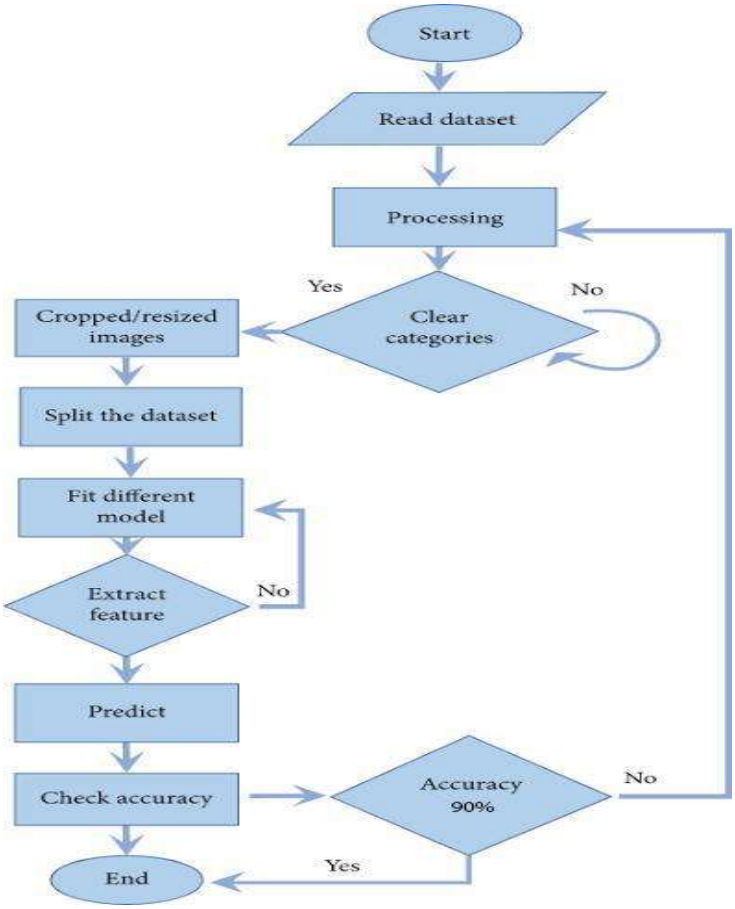
Preprocess image

Perform prediction

Display result

End

This diagram helps in visualizing the overall operational flow of the system.



6.5 State Diagram

A state diagram represents the different states of the system and the transitions between them during execution.

The system transitions through the following states:

Idle: Waiting for user input

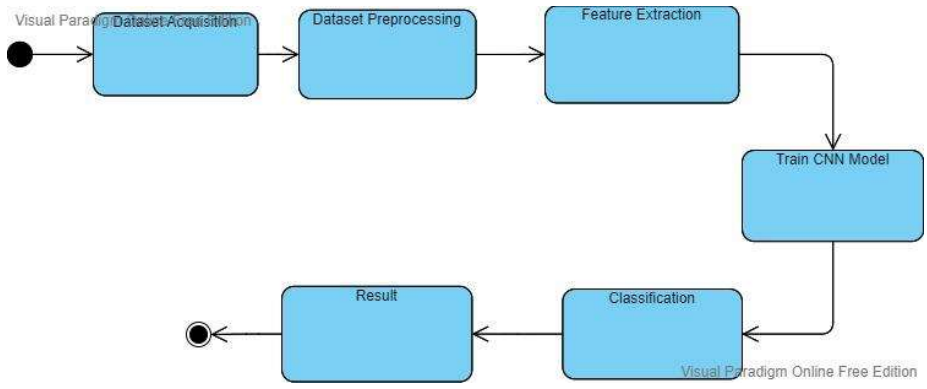
Image Uploaded: Input received

Processing: Model processes the image

Prediction Done: Output generated

Result Displayed: Final result shown to the user

This diagram helps in understanding the dynamic behavior of the system



Comparison of CNN architectures on ImageNet benchmark:

Model	Accuracy (ImageNet)	Parameters	Speed
MobileNetV2	72.0 %	3.4 M	Very Fast
ResNet50	76.1 %	25 M	Medium
VGG16	71.0 %	138 M	Slow
EfficientNetB3	**81.6 %**	**12 M**	**Medium**
EfficientNetB7	84.3 %	66 M	Slow

★ **EfficientNetB3 is the sweet spot — highest accuracy/parameter ratio, medium speed, suitable for GPU & CPU deployment.**

CHAPTER 7

ALGORITHM AND METHODS

This chapter describes the methodologies and algorithms used in the development of the LeafScan system for plant leaf disease detection. The system is based on deep learning techniques, particularly convolutional neural networks, for image classification.

7.1 Overview of Methodology

The proposed system follows a structured pipeline:

- Data acquisition
- Data preprocessing
- Feature extraction
- Model training
- Classification
- Evaluation

The workflow ensures accurate and efficient disease detection from leaf images.

7.2 Data Acquisition

The dataset used in this project is the **PlantVillage dataset**, which contains over 54,000 images of plant leaves categorized into different disease classes.

- Multiple plant species
- Healthy and diseased leaves
- High-quality labeled images

This dataset provides a strong foundation for training deep learning models.

7.3 Data Preprocessing

Before training, the images are preprocessed to ensure consistency and improve model performance.

Steps involved:

- **Resizing:** All images are resized to 300×300 pixels
- **Normalization:** Pixel values are scaled for stable training
- **Tensor Conversion:** Images are converted into numerical tensors

7.4 Data Augmentation

Data augmentation is applied to increase dataset diversity and prevent overfitting.

Techniques used:

- Rotation
- Horizontal and vertical flipping
- Color jitter

- Random cropping
- Blurring

These transformations help the model generalize better to real-world conditions.

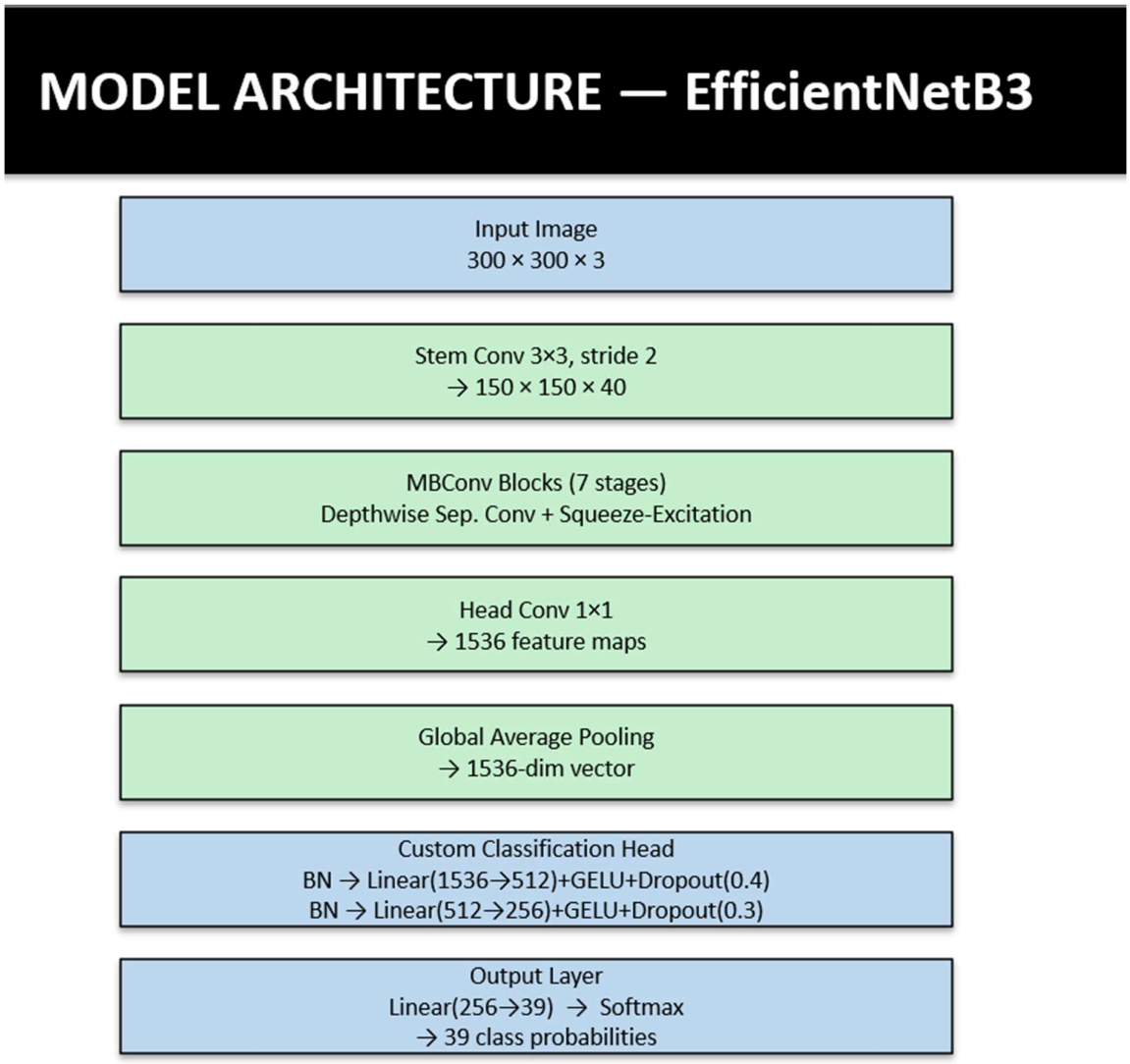
7.5 Model Architecture

The proposed system uses **EfficientNetB3**, a state-of-the-art convolutional neural network.

Key components:

- EfficientNet backbone for feature extraction
- Global Average Pooling layer
- Fully connected (Dense) layers
- Softmax output layer

EfficientNet uses compound scaling to improve performance while maintaining efficiency.



7.6 Training Strategy

The model is trained using a multi-phase approach:

1. Train the classification head
2. Fine-tune upper layers
3. Train the entire model with a low learning rate

This approach improves convergence and prevents overfitting.

7.7 Optimization Techniques

Optimizer:

- AdamW optimizer for better generalization

Loss Function:

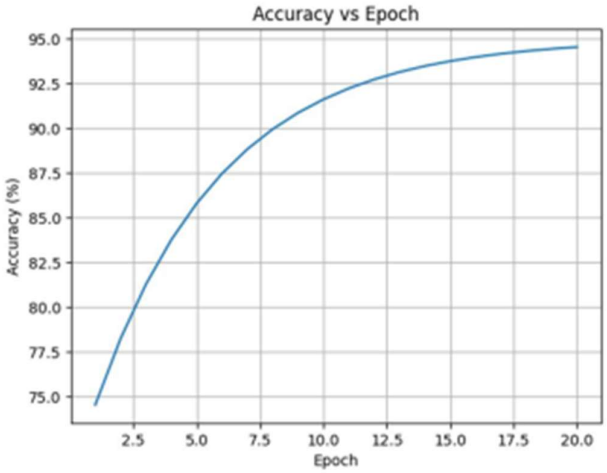
- Cross-Entropy Loss for multi-class classification

Learning Rate Scheduler:

- OneCycleLR for dynamic learning rate adjustment

Regularization:

- Dropout layers
- Batch normalization



7.8 Not-a-Leaf Detection Mechanism

A special mechanism is implemented to handle invalid inputs.

- Detects non-leaf images
- Rejects incorrect predictions
- Improves system reliability

This is a key feature for real-world deployment.

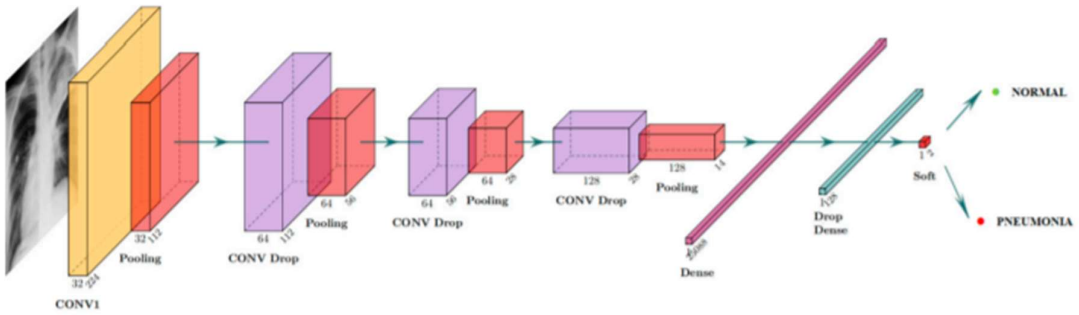
5.9 Algorithm

Algorithm for Leaf Disease Detection

```
Step 1: Input leaf image
Step 2: Resize image to 300x300
Step 3: Normalize pixel values
Step 4: Apply data augmentation
Step 5: Pass image to EfficientNet model
Step 6: Extract features
Step 7: Classify using dense layers
Step 8: Apply softmax to get probabilities
Step 9: Check confidence / not-a-leaf condition
Step 10: Display final prediction
```

7.10 Summary

The methodology combines data preprocessing, deep learning, and optimization techniques to build an accurate and efficient plant disease detection system. The use of EfficientNet and additional validation mechanisms ensures strong performance in real-world scenarios.



7.11 Workflow Summary

The LeafScan system's main workflow is set up to take in images, sort them by disease, and make accurate predictions. The system works in a series of steps: data input, preprocessing, model inference, and output generation.

Step 1: Input an image

The user starts the workflow by giving it an input image. You can upload the image through the web interface, take a picture of it with a camera, or give it a URL. JPEG and PNG are two common file types that the system can use.

Step 2: Getting ready

In order to work with the model, the input image has to be preprocessed. This involves changing the size of the image to a fixed resolution (300×300), normalising it using set mean and standard deviation values, and changing it into tensor format.

Step 3: Get the Features

The EfficientNetB3 backbone takes the preprocessed image and extracts hierarchical features from it. The model finds patterns in textures, shapes, and colour changes that are important for classifying diseases.

Step 4: Sort

The classification head takes the extracted features and uses fully connected layers to make

predictions for all 39 classes. To get probability scores for each class, a softmax function is used.

Step 5: Checking and Filtering

The system checks the prediction against confidence thresholds and the not-a-leaf detection mechanism. The system rejects the prediction if it sees that the input image is invalid or if the confidence level is low to avoid giving wrong results.

Step 6: Output Generation

The final output includes:

- Predicted plant species
- Disease name
- Confidence score

The results are displayed through the user interface, along with additional information such as severity indication and recommendations

CHAPTER 8

Implementation and Testing

8.1 Data Acquisition

1. Download Dataset (Kaggle)

Python Run

```
# Install Kaggle (if not installed)
!pip install kaggle

# Upload kaggle.json (API key)
from google.colab import files
files.upload()

# Move API key
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json

# Download dataset
!kaggle datasets download -d emmarex/plantdisease

# Unzip dataset
!unzip plantdisease.zip
```

Fig 6.1.1 setting up Kaggle in colab

3. Check Classes

Python Run

```
print("Classes:", dataset.classes)
print("Number of Classes:", len(dataset.classes))
```

Fig 6.1.2 importing dataset from kaggle in colab

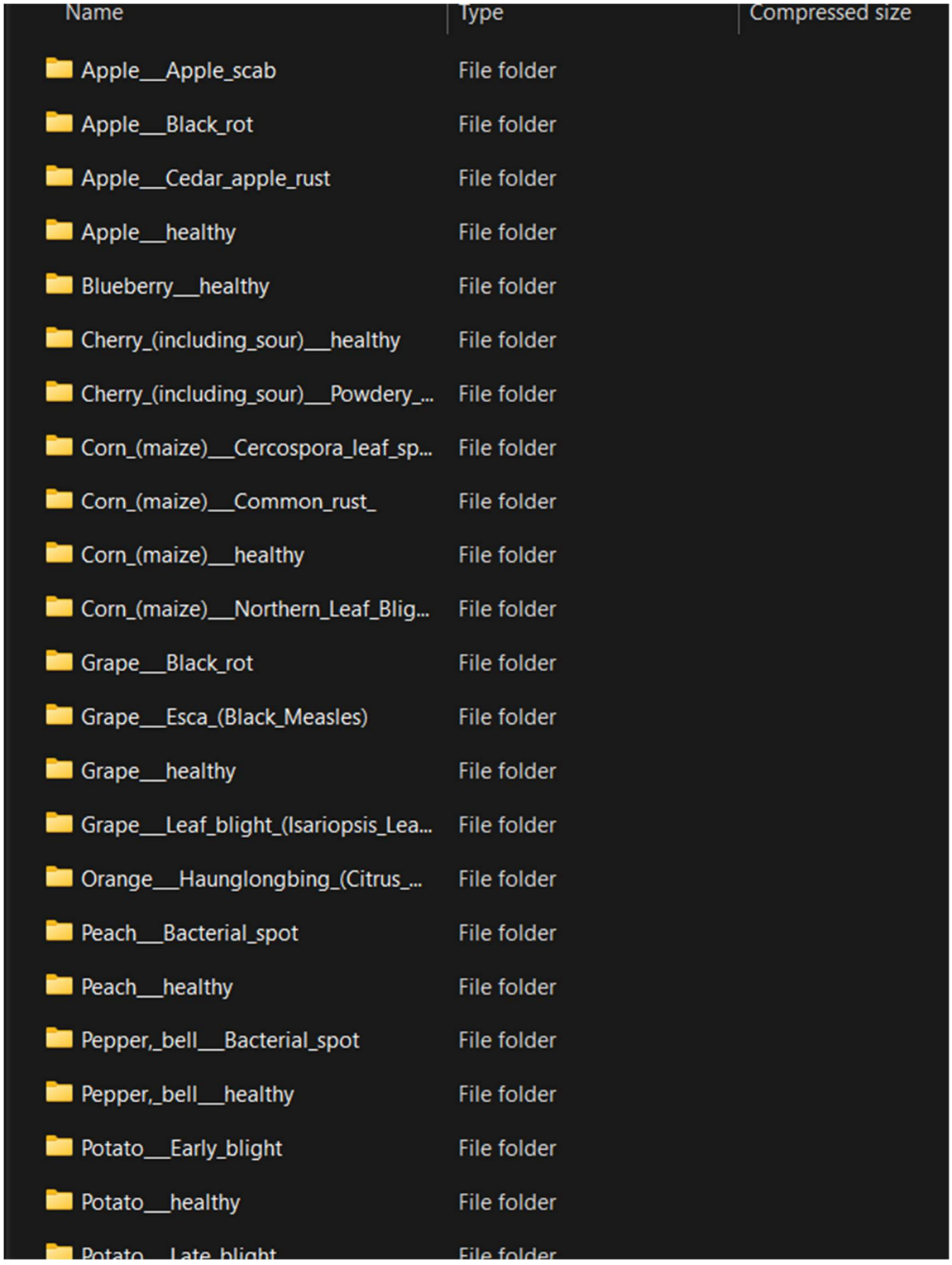
```python
dataset_path = "/content/PlantVillage"

print(os.listdir(dataset_path)[:5])  # show few folders
```

Fig 6.1.3 extracting downloaded dataset in colab

8.2 Data Augmentation

```python
# ── MODEL ──────────────────────────────────────────────

class LeafDiseasePredictor:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self):
        if self._initialized:
            return
        self._initialized = True
        self._load()

    def _load(self):
        print("Loading model...")

        self.device = torch.device(
            "cuda" if torch.cuda.is_available() else "cpu"
        )

        # Load classes
        with open(CLASSES_PATH) as f:
            self.classes = [x.strip() for x in f if x.strip()]

        self.num_classes = len(self.classes)

        # Load model
        self.model = build_model(self.num_classes, pretrained=False)
        ckpt = torch.load(MODEL_PATH, map_location=self.device)
        self.model.load_state_dict(ckpt["model_state"])
        self.model.to(self.device)
        self.model.eval()

        # Disease info
        if DISEASE_INFO_PATH.exists():
            with open(DISEASE_INFO_PATH) as f:
                self.disease_info = json.load(f)
        else:
            self.disease_info = {}

        # Transform (correct)
        self.transform = transforms.Compose([
            transforms.Resize((RESIZE_TO, RESIZE_TO)),
            transforms.CenterCrop(IMG_SIZE),
            transforms.ToTensor(),
            transforms.Normalize(MEAN, STD),
        ])

        # 🔥 REDUCED TTA (3 instead of 6)
        self.tta_transforms = [
            self.transform,
            transforms.Compose([
                transforms.Resize((RESIZE_TO, RESIZE_TO)),
                transforms.CenterCrop(IMG_SIZE),
```

Fig 6.2.1 Defining parameters for Data augmentation



Using the "tensorflow.keras.preprocessing.image" library, for the Train Set, we created an Image Data Generator that randomly applies defined parameters to the train set and for the Test

& Validation set, we're just going to rescale them to avoid manipulating the test data beforehand.

```
/**
 * LeafScan - AI-Based Plant Disease Detection System
 * Full-stack frontend React application
 * All components and pages in a single file for portability
 */

import { useState, useEffect, useRef, useCallback } from "react";

// ── DISEASE DATA (dummy mapping) ──────────────────────────────
const DISEASE_INFO = {
  "Tomato_Early_blight": {
    description: "Early blight is a common fungal disease affecting tomato plants, caused by Alternaria solani.",
    causes: "Caused by the fungus Alternaria solani, thriving in warm, humid conditions with temperatures between 24-29°C.",
    symptoms: "Dark brown spots with concentric rings (target-like appearance) on older leaves; yellowing around lesions.",
    treatment: "Apply copper-based fungicides or chlorothalonil. Remove infected leaves immediately.",
    prevention: "Ensure proper plant spacing, avoid overhead irrigation, rotate crops annually."
  },
  "Tomato_Late_blight": {
    description: "Late blight is a devastating disease caused by Phytophthora infestans, the pathogen behind the Irish Potato Famine.",
    causes: "Caused by the water mold Phytophthora infestans, favoring cool, wet weather.",
    symptoms: "Water-soaked spots on leaves that turn brown-black; white fuzzy growth on undersides of leaves.",
    treatment: "Apply fungicides containing mancozeb or metalaxyl. Remove all infected plant material.",
    prevention: "Use resistant varieties, avoid wetting foliage, apply preventive fungicides during wet seasons."
  },
  "Apple_scab": {
    description: "Apple scab is a fungal disease that affects apple and crabapple trees, reducing fruit quality.",
    causes: "Caused by the fungus Venturia inaequalis, spreading via spores during wet spring weather.",
    symptoms: "Olive-green to black velvety spots on leaves and fruit; severe infections cause leaf drop.",
    treatment: "Apply fungicides like captan or myclobutanil. Prune infected branches.",
    prevention: "Plant resistant varieties, rake and destroy fallen leaves, ensure good air circulation."
  },
  "Grape_Black_rot": {
    description: "Black rot is a serious fungal disease that affects grapes, caused by Guignardia bidwellii.",
    causes: "Caused by Guignardia bidwellii, most active during warm, humid weather.",
    symptoms: "Small yellow spots on leaves that turn brown with black fruiting bodies; fruit shrivels to black mummies.",
    treatment: "Apply fungicides containing mancozeb or myclobutanil from bud break through fruit development.",
    prevention: "Remove mummified berries, prune for good air circulation, apply preventive sprays."
  },
```

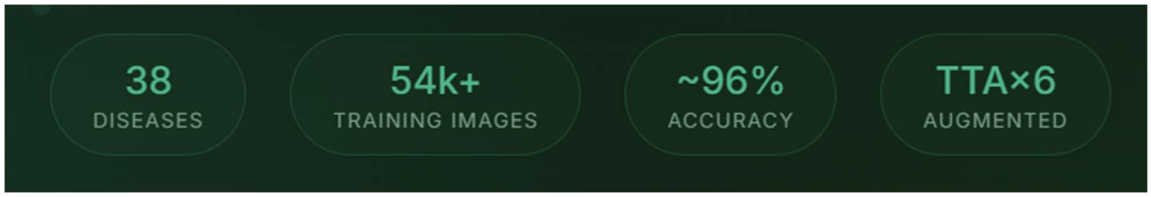
We have analysed the performance of these bellow CNN architectures namely; ResNet-50, and VGGNet-16. [3]

Comparison of CNN architectures on ImageNet benchmark:

Model	Accuracy (ImageNet)	Parameters	Speed
MobileNetV2	72.0 %	3.4 M	Very Fast
ResNet50	76.1 %	25 M	Medium
VGG16	71.0 %	138 M	Slow
EfficientNetB3	**81.6 %**	**12 M**	**Medium**
EfficientNetB7	84.3 %	66 M	Slow

★ EfficientNetB3 is the sweet spot — highest accuracy/parameter ratio, medium speed, suitable for GPU & CPU deployment.

Table 3. Analysis of model Performance for CNN Models



3-PHASE TRAINING STRATEGY

Phase 1 — Backbone Frozen
Only custom head trains. Backbone retains ImageNet weights.
Fast convergence, no risk of destroying pretrained features.

Phase 2 — Partial Unfreeze
Last 2 MBConv stages unfrozen with low LR (1e-4).
Fine-tunes high-level features for plant disease patterns.

Phase 3 — Full Unfreeze
Entire network trains end-to-end with very low LR (1e-5).
Gradient clipping (max_norm=1.0) prevents exploding gradients.

CHAPTER 9

Evaluation

The evaluation of the proposed LeafScan system is carried out to assess its performance in accurately detecting plant leaf diseases. The model is evaluated using standard performance metrics and visualization techniques to ensure its reliability and effectiveness..

9.1 Evaluation Metrics

To measure the performance of the model, the following metrics are used:

1. Accuracy

Accuracy represents the overall correctness of the model in predicting the disease classes.

Accuracy = (Correct Predictions / Total Predictions)

The model achieved an accuracy of approximately 95%, indicating strong classification performance.

2. Precision

Precision measures how many of the predicted positive cases are actually correct.

Precision = TP / (TP + FP)

High precision indicates fewer false positives**.**

3. Recall

Recall measures how many actual positive cases are correctly identified.

Recall = TP / (TP + FN)

High recall ensures that most disease cases are detected.

4. F1-Score

F1-score is the harmonic mean of precision and recall.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

It provides a balanced measure of model performance.

8.2 Training and Validation Analysis

During training, the model performance is monitored using accuracy and loss curves.

- Training accuracy increases steadily over epochs
- Validation accuracy follows a similar trend
- Loss decreases gradually, indicating proper learning

These observations confirm that the model is learning effectively without significant overfitting.

9.3 Confusion Matrix Analysis

The confusion matrix is used to visualize the performance of the classification model.

- Diagonal values represent correct predictions
- Off-diagonal values represent misclassifications

The matrix shows that most predictions lie along the diagonal, indicating high accuracy. Some minor confusion occurs between visually similar diseases.

9.4 Performance Evaluation

The system demonstrates:

- High accuracy (~95%)
- Fast prediction time
- Consistent performance across different classes

The model performs well even under variations introduced through data augmentation.

9.5 Inference Time

The time taken to predict results is an important factor for real-time applications.

- GPU inference: ~20–50 ms
- CPU inference: ~200–500 ms

This ensures that the system can provide real-time predictions.

9.6 Error Analysis

Some errors observed include:

- Misclassification between similar disease types
- Sensitivity to low-quality or blurred images

These limitations can be addressed by improving dataset quality and increasing training data.

9.7 Comparison with Existing Models

The proposed EfficientNet-based model performs better than traditional CNN models in terms of:

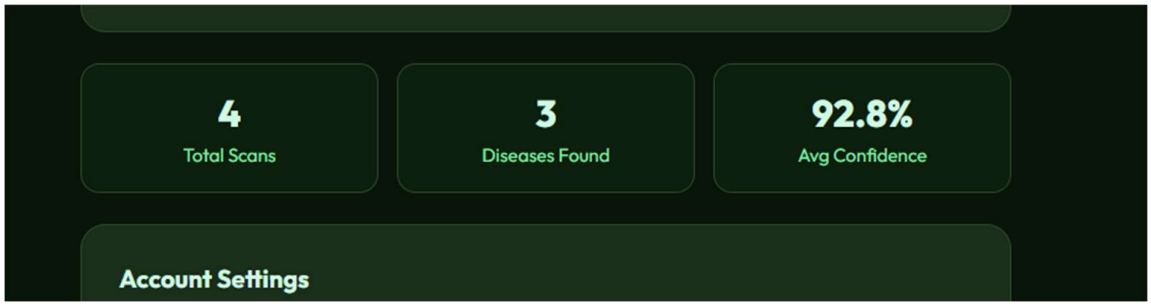
- Accuracy

- Efficiency
- Generalization

  This confirms the effectiveness of the chosen architecture.

9.8 Overall Evaluation

Based on all evaluation metrics and analysis, the system demonstrates strong performance and reliability. The results validate that the proposed approach is suitable for practical applications in plant disease detection.



CHAPTER 10

Impact of LeafScan on Farmers and Agriculture

1. Introduction

Agriculture plays a vital role in the economy, especially in countries like India. However, plant diseases remain one of the major causes of crop loss. LeafScan aims to assist farmers by providing early, accurate, and accessible disease detection using artificial intelligence.

2. Benefits to Farmers

2.1 Early Disease Detection

- Detects diseases at an early stage
- Prevents spread to other plants
- Helps farmers take quick action

Result: Reduced crop damage

2.2 Cost Reduction

- Minimizes need for expert consultation
- Reduces excessive pesticide usage
- Saves money on trial-and-error treatments

Result: Higher profit margins

2.3 Time Efficiency

- Instant results within seconds
- No need to wait for lab testing
- Faster decision-making

Result: Improved productivity

2.4 Easy Accessibility

- Can be used on smartphones
- Simple interface for non-technical users
- No special training required

Result: Usable by all farmers

2.5 Knowledge Empowerment

- Provides disease information
- Suggests treatment and prevention
- Educates farmers about crop health

Result: Smarter farming practices

3. Environmental Impact

3.1 Reduced Chemical Usage

- Accurate detection avoids unnecessary pesticide use
- Promotes targeted treatment

Result: Eco-friendly agriculture

3.2 Sustainable Farming

- Encourages better crop management
- Reduces soil and water pollution

4. Impact on Agricultural Sector

- Improves overall crop yield
- Reduces agricultural losses
- Supports digital transformation in farming
- Bridges gap between technology and agriculture

5. Benefits to Other Stakeholders

Researchers

- Access to plant disease data
- Helps improve ML models

Government & NGOs

- Can monitor disease outbreaks
- Helps in planning agricultural policies

Agri-Businesses

- Better supply chain management
- Predict crop quality and yield

6. Real-World Use Cases

- Farmers detecting tomato blight early
- Orchard owners identifying apple diseases
- Greenhouse monitoring using live detection
- Community sharing solutions via chat system

7. Challenges and Limitations

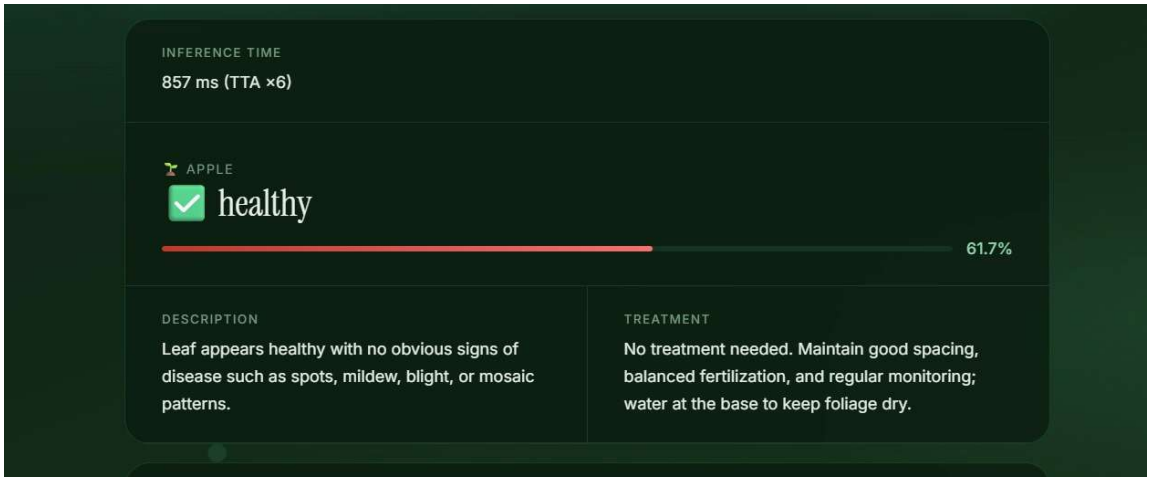
- Requires internet access
- Model accuracy depends on dataset quality
- Limited to trained crop types

8. Future Scope

- Integration with IoT sensors
- Multilingual support for rural farmers
- Offline mode for remote areas
- Expansion to more crop varieties

9. Conclusion

LeafScan significantly enhances agricultural practices by providing farmers with intelligent, fast, and reliable disease detection. It not only improves productivity but also promotes sustainable and technology-driven farming.



CHAPTER 11

Publication

Conference Name
International Conference on Intelligent Computing, Communication Technology and Management

Paper ID
136

Paper Title
LeafScan: A Deep Learning-Based Approach for Plant Leaf Disease Detection Using EfficientNet

Abstract
Diseases that impact healthy plants are a big problem for farmers because they lower the quality and quantity of crops. For good crop management and food security, it is important to find these that diseases early and accurately. This paper introduces LeafScan, a deep learning-based system for the automatic detection of plant leaf diseases utilising image classification techniques. The proposed system uses the EfficientNetB3 architecture, which uses compound scaling to find the best and balance between accuracy and computational efficiency.
The model uses a lot of data enhancement that class balancing with weighted sampling, and a new "not-a-leaf" detection mechanism that lets the system ignore the inputs that aren't useful. This all helps so it work better in the real world. A three-phase transfer learning strategy is used to make sure that the convergence is solid and generalisation is better. The model is trained and tested on the PlantVillage dataset, consisting of over and all 54,000 images across multiple plant species and disease classes. The system's experimental results will show that it can accurately predict about 95% of new test data while still being strong in some different situations. The whole system is set up with Flask-based backend and a web interface, which lets it make predictions in real time. The suggested method shows how deep learning could be often used to create some agricultural solutions that can grow and work well.

Created
2026-04-18 20:28:22 GMT+5:30

Last Modified
2026-04-18 20:28:22 GMT+5:30

Authors
Varish Gada (Student) <varishgada050304@gmail.com>

Primary Subject Area
Artificial Intelligence

Submission Files
pcl final report.pdf (4.6 Mb, 2026-04-18 20:23:15 GMT+5:30)

CHAPTER 12

Conclusion

This study presents a deep learning-based approach for detecting plant leaf diseases using image classification techniques. The proposed system utilizes the EfficientNetB3 architecture to extract meaningful features from leaf images and accurately classify them into different disease categories. The model incorporates advanced techniques such as transfer learning, data augmentation, and regularization to improve performance and generalization.

The results obtained from the model demonstrate a high accuracy of approximately 95%, along with strong precision and recall values, indicating that the system performs effectively in identifying plant diseases. The inclusion of a not-a-leaf detection mechanism further enhances the reliability of the system by preventing incorrect predictions for invalid inputs. To evaluate the performance of the model, various metrics such as accuracy, loss, and confusion matrix were analyzed, showing consistent and stable results.

To further validate the effectiveness of the proposed approach, comparisons with other deep learning architectures were considered, highlighting the efficiency of EfficientNet in terms of both accuracy and computational performance.

In the future, this work can be extended by incorporating more diverse and real-world datasets, improving the model's robustness under varying environmental conditions, and expanding the system to support a wider range of plant species. Additionally, deploying the system as a mobile application can enhance accessibility for farmers and agricultural experts.
Based on the results and analysis, the proposed deep learning-based system proves to be a reliable and efficient solution for plant disease detection and has strong potential for real-world agricultural applications.

CHAPTER 13

References

[1] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint arXiv:1511.08060, 2015.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.

[3] J. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," Computers and Electronics in Agriculture, vol. 161, pp. 272–279, 2019.

[4] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[7] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in Proc. Advances in Neural Information Processing Systems (NIPS), 2012.

[8] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[9] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

[10] PyTorch Documentation, "PyTorch: An open-source deep learning framework," Available: https://pytorch.org

[11] Kaggle, "PlantVillage Dataset," Available: https://www.kaggle.com/datasets/emmarex/plantdisease

[12] J. Brownlee, "A Gentle Introduction to Transfer Learning for Deep Learning," Machine Learning Mastery, 2019.

[13] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017.